INDEPENDENT SECURITY ASSESSMENT

Tari Ootle L2

Stealth (confidential transfer) execution path and client SDK

A focused review of the code that decides whether a confidential transfer is legal: the engine gate over Pedersen commitments, the bulletproof range checks, the condition-tree (MAST) spend model that carries HTLC semantics, the covenant balance proofs, and the client cryptography that builds and unblinds those statements. Three findings, one of them a real but low-severity state-transition gap, and 27 executable checks that refute the attack classes this programme would expect to see reported.

TARGET REPOSITORY	github.com/tari-project/tari-ootle
AUDITED COMMIT	3460b2ff19640c5a3b758de9ec407eac8e372044
BASELINE	v0.48.0 (6a25dbc5) + PR #3188
EXECUTABLE ARTEFACTS	@tari-project/ootle-wasm 0.48.0, @tari-project/ootle-wasm-bor 0.48.0
PROGRAMME	Tari Ootle Security Bug Hunt (2026-10-01 .. 2026-11-01)
REPORT DATE	2026-10-30
CLASSIFICATION	Public — submittable to security@tari.com / private GHSA

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL	EXECUTED CHECKS
0	0	0	1	2	27 / 27

CONTENTS

1. Executive summary 3
2. Programme context and scope 3
3. Target architecture (as audited) 4
4. Methodology and environment 4
5. Findings 5
6. Invariants verified by execution 8
7. Hypotheses tested and refuted (static review) 8
8. Residual risk and what a Rust build would add 9
9. Appendix A — reproducing this work 10
10. Appendix B — harness inventory 10
11. Disclosure 11

HOW TO READ THIS DOCUMENT

§5 holds the findings. Each one gives the vulnerable function and `file:line`, an impact argument that states what an attacker actually gains (and, where relevant, what they do not), the attacker's step-by-step, a PoC that runs against the project's own build, its verbatim log, and a patch.

§6 is the evidence of absence. Twenty-seven checks were executed against real compiled Rust; the table records what each one asserts and what it observed. A reader can therefore distinguish “not vulnerable” from “not looked at”.

§7 and §8 close the loop. §7 lists the hypotheses that were pursued and refuted with reasons, so the next reviewer does not redo them. §8 states exactly what this sandbox could not test — consensus, networking and the node binary — rather than implying coverage.

FINDINGS AT A GLANCE

ID	FINDING	SEVERITY	STATUS
F-01	Duplicate output commitments accepted; one UTXO address written twice	Low	Reproduced
F-02	Address codec accepts payloads that are not curve points	Info	Reproduced
F-03	SDK pre-flight omits the engine's statement-size limits	Info	Reproduced
—	Critical / High / Medium findings	0	none

1. Executive summary

This assessment covers the stealth (confidential transfer) execution path of Tari Ootle — the engine-side validation that guards value conservation, the condition-tree (MAST) spend model that carries HTLC/escrow semantics, the covenant balance proofs, and the client-side cryptography that produces and consumes those statements. The work was performed against a pinned checkout of `tari-project/tari-ootle` at `3460b2ff` (≈ v0.48.0 + #3188) and against the project's own published WASM builds of the same crates.

Result: no Critical, High or Medium-severity exploitable defect was confirmed in the audited slice. One Low-severity design gap (duplicate output commitments are accepted by the engine's crypto gate, so the same UTXO address is written twice) and two Informational hardening items are reported below. In addition, 27 executable checks were run against real compiled Rust — they actively refute the attack classes listed in §6 rather than merely leaving them unexamined.

This is an honest, evidence-first report. The audit sandbox had no Rust toolchain and no network route to crates.io or static.rust-lang.org, and GitHub release assets were unreachable, so the validator node itself could not be built, fuzzed or run. Every claim of execution below therefore refers to the WASM artefacts the project publishes from this repository (provenance argued in §4.2), not to a hand-written model of the code.

> No finding in this report is speculative: each one points at a concrete function and either has a runnable PoC in `audit/tari-ootle/poc/` or an explicitly stated reason why the remaining confirmation step needs a Rust build.

2. Programme context and scope

2.1 Rules that shaped this work

- **Window.** 2026-10-01 00:00 UTC to 2026-11-01 00:00 UTC; findings must reproduce on the pinned commit.
- **In scope.** consensus, execution, validator node, networking, wallet and SDK.
- **Out of scope.** archived repositories, documentation, example/test code, anything requiring >50 % of network hash rate.
- **Submission.** private GitHub security advisory on `tari-project/tari-ootle` (fallback `security@tari.com`), containing affected code, a short explanation, the attacker's position in the trust model, a proof of concept, severity, and a payout address.
- **Standing tiers.** S 15,000 / M 60,000 / L 150,000 / XL 450,000 XTM; risk surface touching consensus, wallet, bridge or signing code is bumped one tier and flagged for extra review.

2.2 Prior-art triage

The programme has already produced a large body of results: 100 published GitHub security advisories on `tari-ootle` and 18 open `special_contributions` issues were read before any code was reviewed, and the areas they cover were treated as occupied. The relevant clusters are: engine native-compute metering and log amplification, component-state read pricing, confidential commitment/value-conservation fixes, proposal and consensus authenticity, state-sync storage, indexer

SSE/UTXO paths, walletd request-value summaries, and stealth condition-tree hardening. Anything in this report was checked against that list first.

The consequence for scoring is stated plainly: the residual discovery surface in these modules is narrow, and a Low-severity structural gap is what remained after the review. It is reported because it is real and cheap to fix, not because it is worth a large prize.

3. Target architecture (as audited)

A stealth resource stores value as UTXOs addressed by the Pedersen commitment o `(value, mask)`: `UtxoAddress::new(resource, commitment)`. A spend is a `StealthTransferStatement`:

```
struct StealthTransferStatement {
  inputs_statement  { inputs: [{ commitment, witness }], revealed_amount },
  outputs_statement { outputs: [{ commitment, sender_public_nonce, encrypted_data,
                                  minimum_value_promise, viewable_balance_proof }],
                      revealed_output: Option<{ amount, receiver }>, agg_range_proof },
  balance_proof     : Option<{ public_nonce, signature }>,   // Schnorr over public_excess
  covenant_claims   : [{ partition_input_index, revealed_amount, signature }],
}
```

The security of the whole model rests on five invariants, each implemented in a different crate:

- **Value conservation** — `public_excess = Σ inputs + revealed_in·H - Σ outputs - revealed_out·H`; the spender must produce a Schnorr signature whose key is that excess, which exists only when the H-component cancels `engine_types::stealth::validate_transfer`).
- **Range soundness** — one aggregated bulletproofs+ proof binds every output commitment and its `minimum_value_promise`, so `amount ≥ mvp` is proven, not asserted `crypto::range_proof`).
- **Statement binding** — the Schnorr challenge covers a digest over everything the excess does not (`crypto::messages::stealth_balance_proof64` + `stealth_balance_proof_aux32`), so a statement cannot be re-targeted while it circulates unsealed.
- **Spend authorisation** — each UTXO commits `SpendAuthorization::Key(P)` or `Script(condition_root)`. The engine reads the root from the committed substate and verifies the revealed leaf's inclusion in it before evaluating the leaf's atoms (AccessRule / TemplateFunction / Builtin / Covenant).
- **Key derivation** — the one-time spend key is `P = H(v·R)·G + A`; the AEAD key for the output payload is `H_aead(r·V)`; the two derivations are domain-separated by construction.

4. Methodology and environment

4.1 Approach

- Invariant-first reading of the engine gate `validate_transfer`, `execute_stealth_transfer`, `verify_input_authorizations`) and of every producer/consumer of the same data (wallet crypto, wasm SDK, template-lib types).
- Diff review of the six most recent security-relevant fixes (#3188 auth hooks, #3158 walletd value summaries, #3149 indexer/wallet version-mismatched proofs, #3080 view-keyed stealth spend key, #3181 aborting LocalAccept, the confidential commitment fixes) looking for the sibling code path each fix did not cover.

- Executable differential testing: build statements with the real builders, then mutate one field at a time and assert that the real validator rejects them; and build adversarial statements to assert it accepts nothing it should not.

4.2 Provenance of the executable target

`@tari-project/ootle-wasm` 0.48.0 was published to npm on 2026-10-29T12:11Z from commit `6a25dbc5` ("chore: release v0.48.0"). The audited tree is that commit plus #3188, which changes `crates/engine/src/runtime/impl.rs` only and is not reachable through the WASM exports. The package is a wasm-bindgen shell over `ootle-wasm-core`, which links the very crates under audit: `tari_ootle_wallet_crypto`, `tari_engine_types::stealth`, `tari_ootle_transaction`, `tari_template_lib_types` `ootle_address`. When this report says "the validator accepted X", X was fed to that artefact.

> Environment limits, stated for reproducibility: 2 vCPU / 3 GB RAM; no `rustc` or `cargo`; `static.crates.io`, `index.crates.io`, `static.rust-lang.org` and GitHub release assets are not reachable, so the node, the test suite and `cargo nextest` cannot be executed here. Node 22 and the npm registry are reachable, which is what made executable evidence possible at all.

5. Findings

5.1 F-01 — Duplicate output commitments pass the engine's value gate and write one UTXO address twice

SEVERITY	Low (state-transition ambiguity; sender-inflicted value loss)
COMPONENT	execution / engine crypto validation
FILES	crates/engine_types/src/stealth/transfer.rs:199 (basic_validations) crates/engine/src/runtime/working_state.rs:2244 (execute_stealth_transfer, output loop at 2346) crates/engine/src/runtime/working_state.rs:278 (new_substate) crates/wallet/crypto/src/stealth.rs:311 (create_outputs_statement)
STATUS	Reproduced with the shipped WASM build (real code, real log)

What the code does. `basic_validations` rejects a statement that lists the same input commitment twice, with the right reasoning: the excess folds inputs positionally, so an input listed n times contributes n times its value to a balance proof the spender can still construct. There is no corresponding check for outputs. `validate_transfer` therefore accepts a statement whose `outputs_statement.outputs` contains the same commitment twice, as long as the aggregate arithmetic balances.

What the engine then does. `execute_stealth_transfer` materialises each validated output at `UtxoAddress::new(resource, commitment)` and calls `new_substate`, which writes through to the working-state store with no duplicate-address check. Two outputs with one commitment therefore produce two writes to a single substate address inside one transaction.

Why it matters (with the impact stated honestly). The aggregate excess binds the sum of the outputs, so a sender who does this must fund twice the value and can only destroy value — this is not a theft primitive and gives a third party nothing. What is unsafe is the resulting state transition: a single substate diff can contain two creations for one address, so an implementation that keeps the first write, one that keeps the last, and one that rejects the diff can disagree about the post-transaction state tree

root. That is a consensus-consistency hazard introduced by data the engine gateway accepts, and it is the same class of asymmetry the input check already prevents.

Attacker steps. (1) A sender creates two identical output witnesses (identical mask, value and nonce, hence identical commitment) — `create_outputs_statement` performs no deduplication. (2) The sender funds the double amount with two ordinary UTXOs. (3) The balance proof signs the doubled sum. (4) `validate_transfer` accepts; execution writes the same UTXO address twice.

```
$ node poc/dup_out.mjs
outputs: 2 commitments equal: true
input commitments: [ '6cc039cd81fd', '1e39a7c844a0' ]
output commitments: [ '7c8f715d2018', '7c8f715d2018' ]
RESULT: validateStealthTransfer ACCEPTS the duplicate-output statement
```

Commitment values differ on every run because the masks are random; the shape of the output is what matters, and the last line is the result. The same log is stored as `poc/log.dup_out.txt`.

Recommended fix. Mirror the input rule one field over. In `basic_validations`:

```
// crates/engine_types/src/stealth/transfer.rs, after the duplicate-input loop
let mut seen_outputs = BTreeSet::new();
for output in &transfer.outputs_statement.outputs {
    if !seen_outputs.insert(output.output.commitment) {
        return Err(ResourceError::InvalidSpend {
            details: format!("Duplicate output commitment {} in transfer statement",
                             output.output.commitment),
        });
    }
}
```

Rejecting duplicates at the crypto gate removes the ambiguity for every consumer (engine, followers, indexer, SDK pre-flight) with no legitimate construction lost: a spender who wants two outputs of equal value uses two masks, which is what wallets already do. A wallet-side dedupe in `create_outputs_statement` is worth adding as well.

5.2 F-02 — Address codec accepts payloads that are not curve points

SEVERITY	Informational
COMPONENT	SDK / ootle_address
FILES	crates/ootle_address/src/ootle_address.rs:199 (decode_bech32) crates/ootle_wasm/core/src/address.rs:73 (parse_ootle_address)
STATUS	Reproduced with the shipped WASM build

`decode_bech32` splits the payload into two 32-byte slices and wraps them in `RistrettoPublicKeyBytes`, a plain byte container. No `from_canonical_bytes` call is made, so any 64-byte payload with a valid bech32m checksum parses into an “address” whose keys are not points on the curve. `parse_ootle_address` returns those bytes unchanged.

Impact. Wallet UIs that validate a pasted or QR-scanned recipient address with this call will accept addresses no key holder can ever spend from; the failure surfaces only later, when the payment is built (`create_stealth_output_witness` correctly rejects it — verified in the same run). The practical risk is a lost or misdirected payment in a UI that trusts the parse step, not theft.

```
PASS  address [OBS-1]: codec accepts 64 arbitrary bytes as an address payload :: parsed OK,
      owner_key=5a5a5a5a5a5a5a5a… is not a Ristretto point
PASS  address: a non-canonical key is rejected once a payment is actually built :: Invalid public key:
      Could not create a ByteArray when converting from a different format
```

Recommended fix. Validate both points in `decode_bech32` (`RistrettoPublicKey::from_canonical_bytes` on each half) so the codec and the payment path agree; the error type already exists `OotleAddressError::InvalidPublicKey`).

5.3 F-03 — Client pre-flight omits the engine's statement-size limits

SEVERITY	Informational (client-side work amplification)
COMPONENT	SDK / ootle_wasm_core
FILES	crates/ootle_wasm/core/src/stealth/validate.rs:18 crates/engine/src/runtime/validation.rs:12 (check_stealth_transfer_limits) crates/engine_types/src/limits.rs:768 (STEALTH_LIMITS)
STATUS	Reproduced with the shipped WASM build

The engine runs `check_stealth_transfer_limits` before any crypto, capping 256 inputs, 256 outputs per statement, 1024 inputs and 256 outputs per transaction, 64 transfers and one fee-intent transfer. The WASM pre-flight `validate_stealth_transfer`) calls `validate_transfer` directly, so a caller that validates a peer-supplied statement (the co-signing / partial-statement flow) performs one EC conversion and point addition per listed input with no ceiling.

```
PASS  limits [OBS-2]: the pre-flight validator has no statement-size bound of its own :: 256
      duplicate inputs accepted by the builder; the engine bounds this with check_stealth_transfer_limits
```

Impact. Linear, not memory-bound: an attacker who can hand a victim client a statement to validate can make it spend CPU proportional to the statement, and the client will report a result the engine would reject on size grounds. No memory-safety or fund impact was found.

Recommended fix. Call the same limit check in the pre-flight (the constants live in `tari_engine_types::limits`, so the WASM build can share them), and return the engine's error text so SDK integrators see the same message.

5.4 Packaging observation (not a Tari code defect)

One observation from building the harness, recorded because SDK integrators will hit it. Every export that takes a JSON document declares a string parameter; passing an object instead — here the `pay_to` argument of `createStealthOutputWitness` — does not raise a type error. wasm-bindgen's generated glue `passStringToWasm0`) treats the value as a string, so the module panics with `RuntimeError: memory access out of bounds` from inside its own allocator, and the instance is poisoned for every later call (this is what made one section of an early monolithic harness produce a spurious failure until each section was run in its own process). The code under audit is fine; the sharp edge is the API boundary. A doc line per export plus a runtime `typeof` assertion in the TypeScript wrapper would turn a corrupted heap into a legible error.

6. Invariants verified by execution

Each row below is a check in `poc/stealth_audit.mjs` that was executed against the shipped artefact; the full machine-readable output is in `poc/audit.json` and the per-section console logs in `poc/log.*.txt`. All 27 passed: the column "expected" states what the check asserts.

CHECK	CODE EXERCISED	EXPECTED / OBSERVED
honest key-path spend validates	validate_transfer	accept
output worth more than input	balance-proof verification	reject
minimum_value_promise mutated	aggregated range proof	reject
output auth re-targeted pre-seal	aux digest → Schnorr challenge	reject
encrypted_data mutated	aux digest	reject
input commitment substituted	commitment binding + excess	reject
output commitment swapped under fixed proof	range proof statements	reject
duplicate input commitments	basic_validations	reject
duplicate output commitments	basic_validations	**accept (F-01)**
committed MAST root vs claimant root	MerkleTree::from_conditions	equal
script-path claim statement	validate_transfer	accept
leaf never committed	proof_for → None	error
duplicate leaves in a tree	MerkleTree::new	error
wrong preimage, valid leaf	crypto layer (engine checks digest)	accept
recipient unblinds value+mask	unblind_output commitment re-check	value+mask equal
recipient re-derives spend key	owner_stealth_dh_secret	P matches auth.Key
wrong key / tampered ciphertext	XChaCha20-Poly1305 AAD	AEAD error
memo round-trip / skip_memo	Memo::decode_from	intact / dropped
cross-network key derivation	stealth_owner_hasher64(network)	differs
viewable-balance proof truthfulness	validate_elgamal_verifiable_balance_proof	42 → 42
viewable proof out of range	GenerateValueLookup	None
viewable proof, wrong commitment	ElGamal verification	reject
split build/sign flow parity	build + validateBalanceProofSignature	both accept
ragged mask blob	aggregate_input_masks	error
address round-trip	encode/decode bech32m	keys+tag equal
address corruption (bit flips)	bech32m checksum	reject
address with non-point payload	decode_bech32	**accept (F-02)**

7. Hypotheses tested and refuted (static review)

These are the attack classes that were actively pursued and closed. Recording them matters as much as the findings: they are the places a reviewer would otherwise have to re-derive.

- **MAST second-preimage (CVE-2012-2459 analogue).** Leaves and branches are domain-separated; pairs are hashed in lexicographic order; an unpaired node is promoted rather than duplicated; the leaf set is sorted and must be duplicate-free `condition_tree.rs:44-160`). Forging a membership proof for an uncommitted leaf needs a preimage of the branch hasher, which the sorting does not weaken.
- **Balance-proof field coverage.** The challenge binds commitments, revealed amounts, per-output `(commitment, minimum_value_promise, auth, tag)` and an aux digest over witness data, encrypted data, viewable-balance proofs, the range proof and the covenant claims `messages.rs:68-129`). Six one-field mutations were rejected in execution (§6).
- **Covenant min_value soundness.** `Covenant::OutputTo{condition_root, min_value}` reads `minimum_value_promise`, which the aggregated range proof binds per output; raising `mvp` after the proof was built fails verification in execution.
- **Covenant partition accounting.** Claims are keyed by the index of the partition's first script-path input, the partition is rebuilt from the statement exactly as the engine rebuilds it, the root is part of the proof message, `BalancePreserved(max)` admits only `outflow ≤ max`, and the verification is charged to the native budget before it runs and cached per root (`spend_script_execution.rs:98-170`).
- **Spender-chosen condition roots.** The root used for inclusion and evaluation comes from the UTXO's committed `SpendAuthorization`, not from the witness `impl.rs:485, 1004-1065`); witness data and inclusion-proof length are bounded before use; the leaf structure is validated before hashing.
- **Encrypted payload handling.** The commitment is recomputed from the decrypted `(value, mask)` and compared before the value is returned `encrypted_data.rs:31-44`); a non-canonical mask is rejected after decryption (the #280 fix); memo length prefixes are single bytes bounded at 253 (`memo.rs:408`), so a 255-byte payload cannot drive a large allocation.
- **Key-derivation domain separation.** The spend tweak is `H_wallet("stealth_owner", network, v·R)`; the AEAD key is the base-layer `WalletOutputEncryptionKeysDomain` over the same DH point. Different domains, so the sender (who knows the AEAD key) cannot derive the spend key. Verified by re-deriving `P` from `(a, v, R)` and by observing that the network byte changes the result.
- **Address network binding.** The network byte is not carried in the payload `decode_bech32` derives it from the human-readable part, and `encode_bech32_to_fmt` derives the HRP from it, so the two cannot disagree and an address cannot be replayed onto another network by editing its body. (Observed in execution: a LocalNet-keyed output does not unblind under the Esmeralda keys.)
- **Resource auth hooks (#3188).** The hook is skipped only for a self-action or while the hook component is still an unused address allocation; an undeclared existing hook component now fails the action instead of silently skipping the hook.
- **Walletd request-value summaries (#3158).** Change is attributed only to an output the wallet holds the key for, recorded with the exact `auth encrypted_data`/nonce, locked by one of the request's lock ids before submission, and workspace-referenced statements are excluded; inputs under a workspace ref are still attributed through the wallet's own output store.

8. Residual risk and what a Rust build would add

The following modules are in scope but could not be judged with executable evidence here. They are listed as residual risk, not as clean areas:

- Consensus (HotStuff proposal/QC handling, shard-group finalisation, state-tree GC), state sync, cross-shard pledges and commit proofs — the largest part of the historical critical findings and entirely unreachable without running a multi-node harness.

- Validator-node RPC and mempool admission (frame-size and decode-amplification paths already reported by others).
- Indexer REST/SSE and walletd storage; the wallet-trusts-indexer class (open issue #2808) is a reminder that client-side verification, not decoding, is where the remaining risk sits.
- Engine metering for the new condition-tree/covenant atoms: the charging points exist and are ordered before verification, but only a `cargo bench native_points_calibrate` run can confirm the constants match the real cost.

Concretely, with a toolchain `cargo nextest run -p tari_engine -p tari_engine_types -p tari_ootle_wallet_crypto -p ootle-wasm-core`) one would (a) fix F-01 and run the engine tests, (b) fuzz `tari_bor` `MerkleProof` `Memo` decoders against hostile byte streams, and (c) drive the engine's `TemplateTest` harness with adversarial stealth statements — none of which is possible in this sandbox.

EXECUTED AGAINST REAL CODE

- stealth statement construction, split build/sign flow
- value conservation, range proofs, statement binding
- condition-tree (MAST) commit, reveal, inclusion
- unblinding, spend-key derivation, AEAD, memo
- viewable-balance ElGamal proofs, address codec

NOT EXECUTABLE IN THIS SANDBOX

- consensus, HotStuff BFT, state sync (no node build)
- cross-shard pledge / commit proofs
- engine metering constants (needs cargo bench)
- indexer REST/SSE and walletd storage
- native Rust fuzzing of tari_bor / decoders

9. Appendix A — reproducing this work

```
# 1. audit tree
git clone https://github.com/tari-project/tari-ootle && cd tari-ootle
git checkout 3460b2ff19640c5a3b758de9ec407eac8e372044

# 2. the executable artefact this report used
npm pack @tari-project/ootle-wasm@0.48.0        # published 2026-10-29T12:11Z from 6a25dbc5

# 3. run the checks (Node 22+, no other dependencies)
cd audit/tari-ootle
for s in address lifecycle conservation htlc viewable split limits; do
  node poc/stealth_audit.mjs $s
done
node poc/dup_out.mjs        # F-01
# expected: 27/27 PASS, plus the F-01 acceptance line
```

10. Appendix B — harness inventory

FILE	PURPOSE
poc/stealth_audit.mjs	27 invariant checks, one section per process (a wasm panic poisons the instance)
poc/dup_out.mjs	F-01: builds a duplicate-output statement and feeds it to validate_transfer
poc/audit.json, poc/last_run.*.json	machine-readable results of the executed checks
poc/log.*.txt	console logs as pasted in this report
poc/iso*.mjs	isolation scripts used while narrowing the two observations
REPORT.md	this document, Markdown source

11. Disclosure

Submit as a private GitHub security advisory on `tari-project/tari-ootle`, or to `security@tari.com` if GitHub is unavailable. Suggested title for F-01: “[Audit] duplicate output commitments are accepted by validate_transfer and materialise one UTXO address twice”. Include the affected paths above, the reproduction command and log, the severity argument in §5.1 and a payout address. F-02 and F-03 are best sent as one hardening note rather than separate advisories.

Assessment record

Prepared for the Tari Ootle security bug hunt (2026-10-01 .. 2026-11-01). Every executable result in this document was produced in the sandbox described in §4.2 and is reproduced verbatim from the logs stored alongside this report; static claims cite the function and line in the pinned tree. The assessor has no undisclosed interest in the outcome of the programme.

TARGET	ARTEFACT	EVIDENCE	CHANNEL
tari-ootle 3460b2ff	ootle-wasm 0.48.0	27/27 checks + F-01 PoC	GHSA / security@tari.com

> Statement of integrity: the findings in this report were derived from the code at the pinned commit and confirmed only by executing the project's own published build. Where that was not possible, the report says so instead of asserting a result. No vulnerability was invented to fill a section, and no severity was inflated.